



IF185301 TESIS

**PENGAMBILAN KEPUTUSAN DAN
PENGATURAN SKALABILITAS ELASTIS PADA
LINGKUNGAN CLOUD YANG HETEROGEN
DENGAN BERBASISKAN PADA PREDIKSI
WORKLOAD**

**MUHAMMAD ZAHID MASRURI
6025222041**

Dosen Pembimbing:

Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.

Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D.

**PROGRAM MAGISTER
BIDANG KEAHLIAN TEKNOLOGI JARINGAN DAN KEAMANAN SIBER
CERDAS (NETICS)
PROGRAM STUDI S2 TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026**



IF185301 TESIS

**PENGAMBILAN KEPUTUSAN DAN
PENGATURAN SKALABILITAS ELASTIS PADA
LINGKUNGAN CLOUD YANG HETEROGEN
DENGAN BERBASISKAN PADA PREDIKSI
WORKLOAD**

**MUHAMMAD ZAHID MASRURI
6025222041**

Dosen Pembimbing:

Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.

Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D.

**PROGRAM MAGISTER
BIDANG KEAHLIAN TEKNOLOGI JARINGAN DAN KEAMANAN SIBER
CERDAS (NETICS)
PROGRAM STUDI S2 TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026**

LEMBAR PENGESAHAN

PROPOSAL TESIS

Judul : Pengambilan Keputusan dan Pengaturan Skalabilitas Elastis pada Lingkungan Cloud yang Heterogen dengan Berbasis pada Prediksi Workload

Mahasiswa : Muhammad Zahid Masruri

NRP : 6025222041

Telah diseminarkan pada,

Hari : Rabu

Tanggal : 30 Juli 2026

Tempat : Ruang 217B

Mengetahui/Menyetujui,

Dosen Penguji:

1.

Dosen Penguji ke-1 (Lengkap)

NIP: 20xxxxxxxxx

2.

Dosen Penguji ke-2 (Lengkap)

NIP: 20xxxxxxxxx

3.

Dosen Penguji ke-3 (Lengkap)

NIP: 20xxxxxxxxx

Dosen Pembimbing:

1.

Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.

NIP: 197708242006041001

2.

Royyana Muslim Ijtihadie, S.Kom., M.Kom.,
Ph.D.

NIP: 197505252003121002

Halaman ini sengaja dikosongkan.

LEMBAR PENGESAHAN TESIS

Tesis disusun untuk memenuhi salah satu syarat memperoleh gelar
Magister Komputer (M.Kom.)
di
Institut Teknologi Sepuluh Nopember

Oleh:
Muhammad Zahid Masruri
NRP: 6025222041

Tanggal Ujian : 30 Juli 2026
Periode Wisuda : September 2026

Disetujui oleh:

Pembimbing:

1. Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.
NIP: 197708242006041001
2. Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D.
NIP: 197505252003121002

Penguji:

1. Dosen Penguji ke-1 (Lengkap)
NIP: 20xxxxxxxx
2. Dosen Penguji ke-2 (Lengkap)
NIP: 20xxxxxxxx
3. Dosen Penguji ke-3 (Lengkap)
NIP: 20xxxxxxxx

Kepala Departemen Teknik Informatika
Fakultas Teknologi Elektro dan Informatika Cerdas

Kepala Departemen Informatika
NIP: 20xxxxxxx

Halaman ini sengaja dikosongkan.

PERNYATAAN ORISINALITAS TESIS

Yang bertanda tangan di bawah ini,

Nama : Muhammad Zahid Masruri (6025222041)
Dosen Pembimbing 1 : Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.
(197708242006041001)
Dosen Pembimbing 2 : Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D.
(197505252003121002)
Program Studi : S2 Teknik Informatika
Departemen : Departemen Teknik Informatika
Fakultas : Fakultas Teknologi Elektro dan Informatika Cerdas

Dengan ini menyatakan bahwa Tesis yang berjudul "Pengambilan Keputusan dan Pengaturan Skalabilitas Elastis pada Lingkungan Cloud yang Heterogen dengan Berbasis pada Prediksi Workload" adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Apabila di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember (ITS), Surabaya, Indonesia.

Surabaya, Juli 2026
Mahasiswa

Meterai 10.000

Muhammad Zahid Masruri
NRP: 6025222041

Dosen Pembimbing 1, Mengetahui, Dosen Pembimbing 2,

Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.
NIP: 197708242006041001

Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D.
NIP: 197505252003121002

Halaman ini sengaja dikosongkan.

Karya ini kupersembahkan kepada:
istriku tercinta, anak-anakku tersayang, dan kedua orang tuaku yang selalu
mendukungku dan mendoakanku.

Halaman ini sengaja dikosongkan.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT yang telah memberikan rahmat serta kemudahan sehingga peneliti dapat menyelesaikan penelitian tesis dengan judul “Pengambilan Keputusan dan Pengaturan Skalabilitas Elastis pada Lingkungan Cloud yang Heterogen dengan Berbasis pada Prediksi Workload” ini dapat terselesaikan.

Peneliti menyadari bahwa tesis ini tidak akan berhasil tanpa adanya bantuan dari beberapa pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D. selaku dosen pembimbing tesis 1 yang telah sabar dalam membimbing dan mengarahkan peneliti sehingga dapat menyelesaikan tesis ini.
2. Bapak Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D. selaku dosen pembimbing tesis 2 yang telah sabar dalam membimbing dan mengarahkan peneliti sehingga dapat menyelesaikan tesis ini.
3. Orang tua dari peneliti yang telah memberikan dukungan, nasihat, kasih sayang, dan perhatian dalam mendidik dan membantu penulis, sehingga dapat menyelesaikan tesis ini.
4. Semua pihak lainnya yang secara langsung maupun tidak langsung dalam membantu peneliti menyelesaikan tesis ini.

Peneliti menyadari bahwa dalam penyusunan laporan tesis ini masih terdapat banyak kekurangan, sehingga kritik dan saran sangat diharapkan. Akhir kata peneliti berharap tesis ini dapat membawa manfaat bagi semua pihak yang menggunakannya.

Surabaya, 13 Mei 2024

Muhammad Zahid Masruri
NRP: 6025222041

Halaman ini sengaja dikosongkan.

ABSTRAK

Pengambilan Keputusan dan Pengaturan Skalabilitas Elastis pada Lingkungan Cloud yang Heterogen dengan Berbasis pada Prediksi Workload

Muhammad Zahid Masruri / NRP 6025222041

Arsitektur cloud modern menghadapi tantangan trade-off antara skalabilitas dan efisiensi biaya. Kubernetes menyediakan kontrol infrastruktur yang baik namun lambat merespons lonjakan lalu lintas, sementara komputasi serverless menawarkan elastisitas tinggi dengan biaya per-request yang lebih mahal. Penelitian ini merancang, mengimplementasikan, dan mengevaluasi arsitektur hybrid yang mengintegrasikan kluster Kubernetes dan serverless dengan prediksi beban kerja berbasis Gated Recurrent Unit (GRU) untuk distribusi lalu lintas yang proaktif.

Sistem yang diusulkan mengimplementasikan pengontrol routing V3 berbasis kapasitas (capacity-driven) yang menggeser lalu lintas secara dinamis antara backend Kubernetes dan serverless berdasarkan pemantauan tail latency. Mekanisme proactive routing menggunakan ekstrapolasi tren beban kerja aktual ($5 \text{ langkah} \times 15 \text{ detik} = 75 \text{ detik ke depan}$) yang dipicu oleh tingkat kepercayaan GRU. Evaluasi dilakukan melalui 30 eksperimen terkontrol yang direplikasi pada empat skenario: K8s+HPA (baseline), Serverless-only, Hybrid-reactive, dan Hybrid-predictive, dengan beban trace ClarkNet ($40 \text{ tahap} \times 30 \text{ detik} = 20 \text{ menit}$, RPS 22–164).

Hasil menunjukkan S4 (Hybrid-predictive) mencapai latensi $p99 = 188 \text{ ms}$ dengan 702 pelanggaran SLO per run, 75% lebih sedikit dibandingkan S3 (Hybrid-reactive) dengan 2.805 pelanggaran (Cohen's $d = 1,21$, large effect). S4 juga 20% lebih murah daripada S1 (\$149/bulan vs \$187/bulan) dengan tingkat kepatuhan SLO 98,4%. Model GRU mencapai 400/400 prediksi sukses dengan tingkat kepercayaan 0,72–0,82 dan latensi inferensi 10–13 ms. Mekanisme proactive routing memicu 9 aksi PREDICTIVE per run, memulai distribusi

ke serverless pada 65% kapasitas K8s.

Kata Kunci: Cloud Computing, Kubernetes, Serverless, Prediksi Beban Kerja, GRU, Distribusi Lalu Lintas, Arsitektur Hybrid

Halaman ini sengaja dikosongkan.

ABSTRACT

Decision Making and Elastic Scalability Management in Heterogeneous Cloud Environments Based on Workload Prediction

Muhammad Zahid Masruri / NRP 6025222041

Modern cloud architecture faces a trade-off between scalability and cost efficiency. Kubernetes provides strong infrastructure control but is slow to respond to traffic spikes, while serverless computing offers high elasticity at a higher per-request cost. This research designs, implements, and evaluates a hybrid architecture integrating Kubernetes and serverless clusters with Gated Recurrent Unit (GRU)-based workload prediction for proactive traffic distribution.

The proposed system implements a V3 capacity-driven routing controller that dynamically shifts traffic between Kubernetes and serverless backends based on tail latency monitoring. The proactive routing mechanism uses actual workload trend extrapolation (5 steps $\times$ 15 seconds = 75 seconds ahead) gated by GRU confidence. Evaluation is conducted through 30 replicated controlled experiments across four scenarios: K8s+HPA (baseline), Serverless-only, Hybrid-reactive, and Hybrid-predictive, using the ClarkNet HTTP trace (40 stages $\times$ 30 seconds = 20 minutes, RPS 22–164).

Results show S4 (Hybrid-predictive) achieves p99 latency = 188ms with 702 SLO violations per run, 75% fewer than S3 (Hybrid-reactive) with 2,805 violations (Cohen’s $d = 1.21$, large effect). S4 is also 20% cheaper than S1 (\$149/month vs \$187/month) at 98.4% SLO compliance. The GRU model achieves 400/400 successful predictions with confidence 0.72–0.82 and inference latency 10–13ms. The proactive routing mechanism triggers 9 PREDICTIVE actions per run, initiating serverless distribution at 65% of K8s capacity.

Keywords: Cloud Computing, Kubernetes, Serverless, Workload Prediction,

GRU, Traffic Distribution, Hybrid Architecture

Halaman ini sengaja dikosongkan.

DAFTAR ISI

LEMBAR PENGESAHAN PROPOSAL TESIS	i
LEMBAR PENGESAHAN TESIS	iii
PERNYATAAN ORISINALITAS TESIS	vi
KATA PENGANTAR	x
ABSTRAK	xii
ABSTRACT	xv
DAFTAR ISI	xviii
DAFTAR TABEL	xxiii
DAFTAR GAMBAR	xxv
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
1.4 Hipotesis Penelitian	5
1.5 Manfaat Penelitian	5
1.6 Kontribusi Penelitian	6
1.7 Batasan Masalah	6
2 KAJIAN PUSTAKA	7

2.1	Cloud Computing	7
2.1.1	Container & Kubernetes	9
2.1.2	Function as a Service and Serverless	10
2.2	Integrasi Antara Kluster Kubernetes dan Serverless	11
2.3	Cloud Application Performance	12
2.3.1	SLA Violation	12
2.3.2	Request Response Time	12
2.3.3	Resource Utilization	13
2.4	Algorithm and Prediction Model	13
2.4.1	LSTM	13
2.4.2	GRU	13
2.5	Kubernetes Scalling Strategy	14
2.5.1	Kluster & Pod Autoscaler	14
2.5.2	Metode ElaX dalam Scalling Kluster	14
2.6	Routing dan Distribusi Trafik pada Arsitektur Hybrid	16
3	METODOLOGI	19
3.1	Studi Literatur	20
3.2	Pengumpulan Data	20
3.3	Perancangan Metode	21
3.4	Implementasi Metode	21
3.4.1	Load Predictor	22
3.4.2	Resource Alocator	22
3.4.2.1	Tuning Koefisien	23
3.4.3	Online Controller	23
3.5	Rencana Evaluasi	23
3.5.1	Evaluasi Traffic Predictor	25
3.5.2	Evaluasi Router dan Scalling	25
3.6	Arsitektur Sistem	26
3.7	Algoritma 1: Pengontrol Routing V3	26
3.7.1	Proactive Routing	27

3.8	Algoritma 2: Pengontrol Skalabilitas Cluster	27
3.9	Rancangan Pengujian	27
3.10	Metode Analisis Biaya	28
3.11	Kalibrasi dan Optimasi Parameter	28
3.11.1	Kalibrasi Kapasitas	28
3.11.2	Optimasi Hiperparameter GRU	29
3.11.3	Pencarian Parameter Pengontrol	29
3.11.4	Karakterisasi Beban Kerja dan Pemicu Pelatihan Ulang	30
3.12	Penyusunan Laporan	30
3.13	Rencana dan Jadwal Kegiatan Penelitian	30
4	HASIL DAN PEMBAHASAN	31
4.1	Performa Model Prediksi GRU	31
4.1.1	Optimasi Hyperparameter GRU	32
4.2	Validasi Mekanisme Sistem	33
4.2.1	Perpindahan Bobot Lalu Lintas	33
4.2.2	Proactive Routing	33
4.2.3	Skalabilitas Node Dinamis	33
4.3	Evaluasi Komparatif	34
4.4	Analisis Statistik	35
4.5	Analisis Biaya	35
4.6	Dampak Proactive Routing	36
4.7	Pembahasan	37
4.7.1	Mengapa S1 Memiliki SLO Terendah	37
4.7.2	Mengapa S2 Sangat Mahal	37
4.7.3	S4 sebagai Trade-off Terbaik	38
4.7.4	Keterbatasan	38
5	KESIMPULAN DAN SARAN	39
5.1	Kesimpulan	39
5.1.1	RQ1: Perancangan Prediksi Beban Kerja dengan GRU	39

5.1.2	RQ2: Perancangan Pengambilan Keputusan dengan Modifikasi ElaX	39
5.1.3	RQ3: Evaluasi Mekanisme yang Dimodifikasi	40
5.2	Keterbatasan	41
5.3	Saran untuk Penelitian Selanjutnya	41
DAFTAR PUSTAKA		43
LAMPIRAN		47
LAMPIRAN A: Konfigurasi Eksperimen		49
LAMPIRAN B: Hasil Eksperimen Mentah		51
LAMPIRAN C: Code Listings		53
BIOGRAFI PENULIS		55

Halaman ini sengaja dikosongkan.

DAFTAR TABEL

3.1	Contoh dataset request	21
3.2	Skenario eksperimen	28
3.3	Rencana Kegiatan Penelitian Thesis	30
4.1	Performa model GRU selama eksperimen	32
4.2	Perbandingan konfigurasi GRU sebelum dan sesudah HPO . .	33
4.3	Ringkasan hasil per-skenario (n=5)	34
4.4	Perbandingan statistik S4 vs S3 (n=5)	35
4.5	Proyeksi biaya bulanan (AWS, 30 hari kontinu)	36

Halaman ini sengaja dikosongkan.

DAFTAR GAMBAR

2.1	Cloud services and division of responsibilities (Kavis, 2014) . .	8
2.2	Architecture of a Kubernetes cluster	9
2.3	Serverless Architecture (Mampage et al., 2022)	11
2.4	Arsitektur ElaX	15
2.5	Online Control Algorithm	16
3.1	Alur penelitian tesis	19
3.2	Routing Controller	24
4.1	Perbandingan latensi p99 antar skenario (n=5)	34
4.2	Proyeksi biaya bulanan per skenario	36
4.3	Dampak proactive routing pada pelanggaran SLO S4	37

Halaman ini sengaja dikosongkan.

BAB 1

PENDAHULUAN

Bab ini membahas mengenai hal-hal yang menjadi dasar dari penelitian. Hal tersebut meliputi latar belakang yang membahas mengenai identifikasi masalah penelitian beserta referensi penelitian sebagai penelitian terdahulu, rumusan masalah, tujuan penelitian, batasan masalah, hingga manfaat penelitian.

1.1 Latar Belakang

Arsitektur cloud modern mengubah bagaimana aplikasi web dikembangkan, digunakan, dan dikelola; menawarkan tingkat fleksibilitas, skalabilitas, dan efisiensi yang belum pernah terjadi sebelumnya (Smit, 2013). Evolusi ini ditandai dengan pergeseran dari penyediaan infrastruktur dasar seperti mesin virtual (Virtual Machine atau VM) atau Virtual Private Server (VPS), menjadi lebih skalabel dengan Container, dan edge worker yang tersebar menggunakan model serverless. Ini didorong oleh kebutuhan akan sumber daya komputasi yang lebih efisien, pengurangan biaya operasional, dan peningkatan kinerja serta skalabilitas untuk mengakomodasi kompleksitas dan basis pengguna yang berkembang karena model tradisional kesulitan untuk mengikuti.

Kubernetes adalah sistem manajemen cluster berdasarkan teknologi container. Popularitasnya merupakan hasil dari kemampuan skalabilitas berdasarkan permintaan. Namun, algoritma skalabilitas tradisional yang ditawarkan oleh penyedia layanan cloud tidak dapat memenuhi lonjakan lalu lintas mendadak karena waktu startup container yang lambat, sering kali menyebabkan pen-

ingkatan latensi dan alokasi sumber daya berlebihan yang tidak terpakai ketika lalu lintas menurun (Yang et al., 2019). Penundaan waktu respons aplikasi dapat merusak pengalaman pengguna, akibatnya menyebabkan kerugian dalam bisnis.

Bersamaan dengan Kubernetes, komputasi Serverless, atau Function as a Service (FaaS) telah mendapatkan traksi dalam beberapa tahun terakhir. Model ini menangani penyediaan dan scaling infrastruktur secara otomatis, memungkinkan aplikasi dapat berjalan dengan tingkat ketersediaan tinggi tanpa kerumitan dalam pengelolaan server. Sehingga pengembang dan organisasi dapat sepenuhnya fokus pada layanan aplikasi (Sadaqat et al., 2018; Savage, 2018; Sewak and Singh, 2018). Selain itu, berdasarkan benchmark Yu et al. (2020) fungsi serverless dapat diskalakan dari nol ke kapasitas target dalam hitungan detik, dengan rata-rata waktu cold start dari 300 ms hingga 8 detik untuk sebagian besar platform tergantung pada runtime yang digunakan.

Dalam konteks evolusi arsitektur cloud yang terus berkembang, muncul tantangan signifikan yang melibatkan tradeoff antara Skalabilitas dan Efisiensi, di mana upaya mencapai skalabilitas tinggi untuk menanggulangi lonjakan lalu lintas yang tak terprediksi sering kali bertentangan dengan efisiensi penggunaan sumber daya untuk mengoptimalkan biaya. Strategi skalabilitas tradisional, seperti over-provisioning kluster Kubernetes, dengan cara mengalokasikan sumber daya berlebih pada sebuah kluster, untuk menangani lonjakan lalu lintas yang tidak terprediksi, mengakibatkan penggunaan sumber daya yang kurang optimal ketika tidak terdapat lalu lintas tinggi, meningkatkan biaya tanpa meningkatkan performa (Zhang et al., 2021).

Untuk mengevaluasi dan mengoptimalkan aplikasi cloud secara efektif, beberapa metrik digunakan. Tail latency, mengukur waktu yang diperlukan untuk memproses permintaan yang paling lambat dalam sistem, digunakan untuk memahami kinerja terburuk dan memastikan bahwa semua permintaan pengguna memenuhi waktu respons yang dapat diterima (Aslanpour et al., 2020). Running time, durasi total untuk menjalankan tugas atau fungsi ter-

tentu. Alokasi sumber daya yang efektif, termasuk distribusi CPU, memori, dan penyimpanan, digabungkan dengan penggunaan CPU di seluruh cluster membantu mengidentifikasi kemungkinan ketidakefisienan sumber daya.

Perkembangan terbaru dalam komputasi awan telah memperkenalkan berbagai metode untuk mengatasi masalah latensi dan skalabilitas pada arsitektur Kubernetes dan Serverless. Yang et al. (2019) memanfaatkan algoritma pembelajaran mesin berbasis LSTM (Long Short-Term Memory) untuk melakukan peramalan beban kerja sebagai bagian dari algoritma yang diusulkan. Hasilnya menunjukkan kurang dari 18% sumber daya yang mengalami over-provisioning. Yuan and Liao (2024) memanfaatkan peramalan deret waktu untuk mengantisipasi kebutuhan sumber daya, secara signifikan memangkas waktu cold start serverless. Sementara itu, He (2020) menggunakan algoritma prediktif untuk mengoptimalkan jumlah instansi sebelum terjadi fluktuasi permintaan. Mondal et al. (2023) memanfaatkan model GRU (Gated Recurrent Unit) untuk memprediksi beban kerja Kubernetes secara custom.

LSTM (Long Short-Term Memory) adalah jenis jaringan saraf dalam yang dirancang khusus untuk mengatasi masalah pada data deret waktu. LSTM sangat berguna untuk kasus-kasus prediksi di mana data memiliki dependensi temporal yang panjang, seperti dalam prediksi beban kerja server cloud. Selain LSTM, terdapat juga Gated Recurrent Unit (GRU) yang memiliki struktur lebih sederhana dan memerlukan sumber daya lebih sedikit dibandingkan LSTM, namun tetap mampu memberikan performa prediksi yang tidak identik (Mondal et al., 2023). GRU memilih informasi yang relevan untuk disimpan atau dibuang, menghasilkan prediksi yang lebih efisien dalam konteks komputasi awan.

Untuk membuat model prediksi dalam cloud computing yang efektif, diperlukan dataset yang menggambarkan kondisi sistem. Yang et al. (2019) menggunakan dataset berupa volume request dari Calgary Trace dan ClarkNet dalam mengembangkan bagian traffic predictor pada metode scalling yang diusulkan. Calgary Trace menyediakan data lalu lintas jaringan dari Universi-

ty of Calgary, berisi lebih dari 4 juta request HTTP. ClarkNet dataset berfokus pada penggunaan web, menyediakan data rinci tentang permintaan HTTP ke server WWW ClarkNet.

Penelitian ini mengusulkan strategi distribusi lalu lintas antara kluster Kubernetes dan serverless, bertujuan untuk meningkatkan efisiensi biaya dan mengurangi waktu respons selama lonjakan lalu lintas jaringan yang tidak terduga. Berdasarkan metode Elax, studi ini akan menggabungkan Prediktor Beban Kerja berbasis GRU untuk peramalan beban kerja yang tepat, ditambah dengan mekanisme Reservasi Sumber Daya untuk alokasi sumber daya yang optimal. Sebuah Pengontrol akan melepaskan sumber daya yang tidak terpakai berdasarkan umpan balik dengan tetap memastikan persyaratan tail-latency terpenuhi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, terdapat beberapa rumusan masalah yang dirumuskan sebagai berikut:

1. Bagaimana merancang prediksi beban kerja HTTP menggunakan Gated Recurrent Unit (GRU) untuk aplikasi cloud?
2. Bagaimana merancang pengambilan keputusan dengan memodifikasi kerangka kerja Elax untuk skalabilitas dan distribusi lalu lintas pada kluster heterogen (Kubernetes dan serverless)?
3. Bagaimana mengevaluasi mekanisme Elax yang dimodifikasi untuk automatic scaling dan load distribution pada integrasi Kubernetes dan serverless?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah dijelaskan, tujuan besar dari penelitian tugas akhir ini adalah sebagai berikut:

1. Menganalisis dampak integrasi Kubernetes dan serverless dalam men-

gatasi tantangan efisiensi dan scalability aplikasi.

2. Menilai performa integrasi Kubernetes dan Serverless dalam meningkatkan kemampuan Scalling aplikasi sehingga bisa mengurangi masalah Latency dan Over-provisioning.
3. Mengukur dan mengevaluasi pendekatan ini dibandingkan dengan model lain dalam menangani permasalahan latency dan over-provisioning.
4. Mengembangkan metode scalling dan routing traffic yang mengintegrasikan Kubernetes dan Serverless.

1.4 Hipotesis Penelitian

Berdasarkan rumusan masalah dan tinjauan pustaka, penelitian ini mengajukan tiga hipotesis:

1. H1 (Efisiensi biaya hybrid): Arsitektur hybrid dengan distribusi lalu lintas cerdas memberikan efisiensi biaya yang lebih baik dibandingkan pendekatan murni (Kubernetes-only atau serverless-only) pada tingkat kepatuhan SLO yang setara.
2. H2 (Prediktif > Reaktif): Mekanisme proactive routing berbasis prediksi GRU menghasilkan lebih sedikit pelanggaran SLO dibandingkan mekanisme reactive routing murni.
3. H3 (Adekuasi GRU): Model GRU memberikan prediksi yang memadai (kepercayaan $\geq 0,5$, latensi < 50 ms) untuk mendukung keputusan routing proaktif pada beban kerja HTTP.

1.5 Manfaat Penelitian

Adapun manfaat yang bisa diperoleh dari penelitian ini dipaparkan sebagai berikut:

1. Memahami bagaimana dynamic scaling and routing dengan menggabungkan Kubernetes dan Serverless dapat meningkatkan kemampuan Scalling dan efisiensi pada aplikasi.

2. Memberikan pemahaman dan wawasan tentang bagaimana dynamic scaling and routing dengan mengintegrasikan Kubernetes dan Serverless bisa mengatasi tantangan scaling dan efisiensi pada aplikasi.

1.6 Kontribusi Penelitian

Adapun kontribusi yang diperoleh dari penelitian ini sebagai berikut:

1. Merancang metode prediksi beban traffic pada aplikasi dengan mempertimbangkan history volume permintaan per detik.
2. Membuat sebuah metode scaling dan dynamic routing dengan menggabungkan Kluster Kubernetes dan Serverless yang berbasis metode Elax.

1.7 Batasan Masalah

Agar penelitian yang dilakukan dapat berjalan sesuai dengan tujuan yang dipaparkan, ditetapkan beberapa batasan masalah sebagai berikut:

1. Workload: fib(33) CPU-bound + 50 ms I/O wait, dengan trace HTTP ClarkNet yang direplikasi melalui k6.
2. Infrastruktur: kluster k3d pada satu mesin 16-inti AMD, HAProxy untuk routing, Knative Serving dengan KPA.
3. Metrik: latensi p99 (ambang SLO 200 ms), pelanggaran SLO, proyeksi biaya bulanan (AWS EKS+EC2+Lambda).
4. Tidak ada batasan CPU pada level pod; batas CPU pada level node Docker (`--cpus`) adalah batas yang sebenarnya.
5. Basis metode dan algoritma yang digunakan adalah Elax dengan modifikasi pengontrol routing V3 berbasis kapasitas dan mekanisme proactive routing.

BAB 2

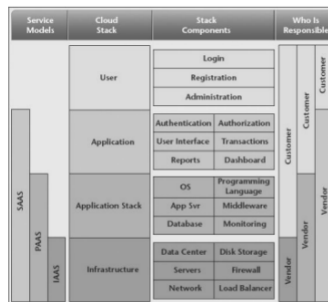
KAJIAN PUSTAKA

Pada bab ini dijelaskan mengenai pustaka yang digunakan sebagai landasan ilmiah penelitian dan pustaka penelitian yang telah dilakukan sebelumnya terkait penelitian yang akan dilakukan.

2.1 Cloud Computing

Cloud computing, menurut National Institute of Standards and Technology (NIST), adalah model yang menyediakan akses on-demand ke kumpulan jaringan komputasi yang dapat dikonfigurasi. Model ini mencakup tiga layanan utama: Infrastructure-as-a-Service (IaaS) yang menawarkan sumber daya virtual yang dapat diskalakan; Platform-as-a-Service (PaaS) yang menyediakan alat dan middleware untuk pengembang; dan Software-as-a-Service (SaaS) yang menyediakan aplikasi perangkat lunak melalui internet (Mell and Grance, 2011). Peralihan dari komputasi tradisional ke model berbasis layanan ini didorong oleh kemajuan dalam virtualisasi, penyimpanan, dan daya pemrosesan (Malathi, 2011).

Teknologi kontainer, yang didukung oleh platform seperti Docker, memudahkan pengelolaan kontainer dan membuka jalan bagi alat orkestrasi seperti Kubernetes (Yadav et al., 2019). Kubernetes telah merevolusi orkestrasi kontainer dengan mengotomatiskan penerapan, penskalaan, dan manajemen aplikasi yang terkontainerisasi (Pahl et al., 2019). Selain itu, model komputasi tanpa server seperti Function as a Service (FaaS) lebih lanjut menyederhanakan manajemen infrastruktur, memungkinkan pengembang fokus pada

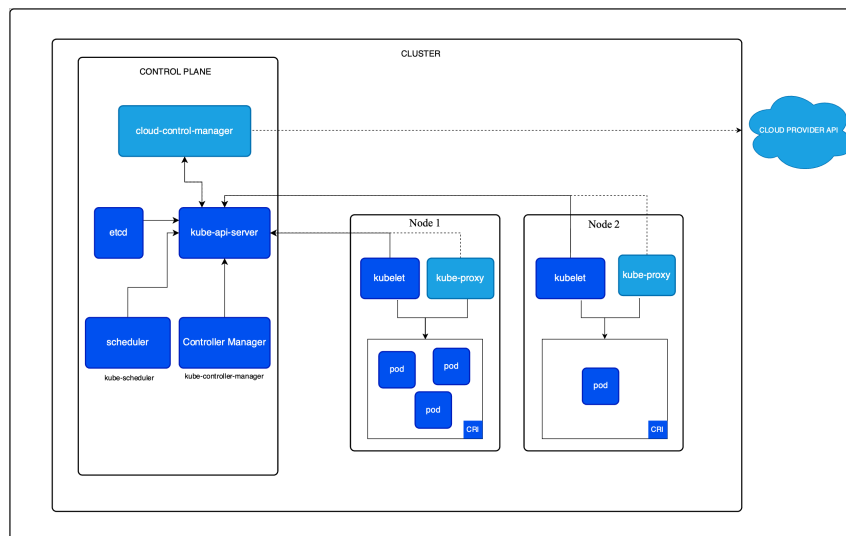


Gambar 2.1 Cloud services and division of responsibilities (Kavis, 2014)

penulisan kode (Wurster et al., 2018).

2.1.1 Container & Kubernetes

Kontainer telah menjadi teknologi utama dalam komputasi awan, menawarkan alternatif yang lebih efisien dibandingkan mesin virtual tradisional (VM). Berbeda dengan VM yang memerlukan sistem operasi lengkap, kontainer berbagi kernel dengan sistem host dan mengisolasi lingkungan eksekusi aplikasi, mengurangi overhead dan meningkatkan kinerja. Isolasi ini dicapai melalui namespace dan control group—fitur Linux yang dapat membatasi akses dan penggunaan sebuah proses, memungkinkan multi-tenancy yang aman dan kontrol alokasi sumber daya. Docker, menyediakan platform yang menyederhanakan manajemen package, distribusi, dan manajemen aplikasi yang terkontainerisasi.



Gambar 2.2 Architecture of a Kubernetes cluster

Seiring dengan meluasnya penggunaan kontainer, kebutuhan akan orkestrasi mereka memunculkan Kubernetes. Kubernetes adalah platform open-source yang dirancang untuk mengotomatisasi penerapan, penskalaan, dan operasi kontainer aplikasi. Platform ini mengelompokkan kontainer yang membentuk aplikasi ke dalam unit logis untuk memudahkan manajemen dan penemuan. Kubernetes menyediakan kerangka kerja untuk menjalankan sistem terdis-

tribusi secara andal, dengan penemuan layanan dan penyeimbangan beban, orkestrasi penyimpanan, pengguliran otomatis dan pembatalan pengguliran, serta kemampuan penyembuhan diri.

Komponen utama dari control plane meliputi:

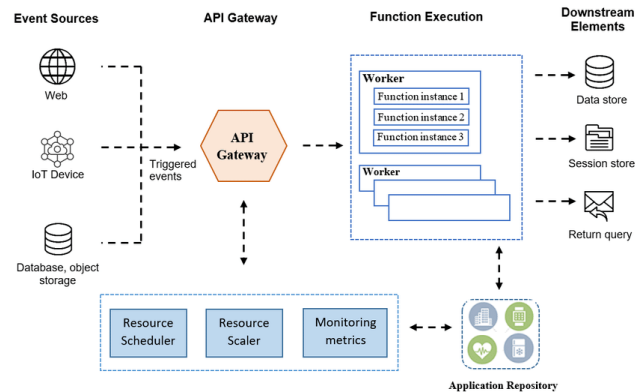
- etcd: Penyimpanan key-value yang persisten, ringan, dan terdistribusi, menyimpan data konfigurasi, status, dan metadata kluster.
- kube-api-server: Server API yang berfungsi sebagai antarmuka depan untuk control plane.
- kube-scheduler: Komponen yang memantau pod baru yang belum memiliki node dan memilih node berdasarkan kriteria penjadwalan.
- kube-controller-manager: Menjalankan proses kontrol yang mengatur status kluster.
- cloud-control-manager: Menghubungkan kluster dengan API penyedia cloud.

Pada tingkat node, komponen utama meliputi:

- kubelet: Agen yang berjalan di setiap node yang memastikan kontainer berjalan dalam pod.
- kube-proxy: Proxy jaringan yang menjaga aturan jaringan untuk komunikasi pod.
- Container Runtime Interface (CRI): Antarmuka untuk runtime kontainer agar terintegrasi dengan kubelet.
- Pod: Unit terkecil yang dapat dibuat dan dikelola di Kubernetes.

2.1.2 Function as a Service and Serverless

Function as a Service (FaaS) atau serverless, merupakan model layanan cloud yang memungkinkan pengembang untuk menerapkan fungsi individu—blok kode yang dipicu oleh peristiwa tertentu. FaaS menyembunyikan infrastruktur dasar, termasuk server, sistem operasi, dan bahkan lingkungan eksekusi aplikasi, sehingga pengembang dapat fokus hanya pada penulisan logika bisnis.



Gambar 2.3 Serverless Architecture (Mampage et al., 2022)

Dalam arsitektur serverless, penyedia cloud bertanggung jawab untuk menjalankan fungsi sebagai respons terhadap peristiwa dan sepenuhnya mengelola infrastruktur yang mendasarinya. Pelanggan dikenakan biaya berdasarkan jumlah sumber daya yang benar-benar digunakan oleh aplikasi mereka, bukan berdasarkan unit kapasitas yang telah dibeli sebelumnya. Model ini dapat menghasilkan penghematan biaya yang signifikan karena menghilangkan kebutuhan untuk menjalankan kapasitas server yang tidak digunakan secara terus-menerus, namun akan menjadi mahal apabila terdapat beban konstan yang memerlukan sumber daya secara terus menerus.

2.2 Integrasi Antara Kluster Kubernetes dan Serverless

Seiring dengan pertumbuhan komputasi serverless, interaksi antara Function as a Service (FaaS) dan Kubernetes menjadi semakin relevan. Sementara Kubernetes mengorkestrasi kontainer dengan efisien, komputasi serverless dapat menangani lonjakan peristiwa dengan cepat tanpa perlu mengelola server secara langsung. Mengintegrasikan keunggulan Kubernetes dan FaaS memungkinkan mekanisme distribusi lalu lintas yang secara dinamis dapat menyesuaikan skala dengan permintaan beban kerja, sambil mengoptimalkan biaya dan kinerja.

Strategi integrasi melibatkan konfigurasi deployment di mana Kubernetes mengelola layanan jangka panjang yang konstan dan predictable, sementara

fungsi serverless menangani tugas-tugas singkat ketika layanan kubernetes belum melakukan scalling sehingga tidak dapat dilayani. Model hibrida ini berpotensi menawarkan yang terbaik dari kedua dunia: kapabilitas orkestrasi dan manajemen Kubernetes serta model on-demand dan pay-as-you-go dari serverless.

2.3 Cloud Application Performance

Menentukan seberapa layanan cloud memenuhi ekspektasi pengguna dan Service Level Agreements (SLA). Untuk memastikan bahwa aplikasi cloud beroperasi secara optimal, beberapa metrik kunci digunakan untuk mengevaluasi berbagai aspek kinerja.

2.3.1 SLA Violation

Pelanggaran SLA adalah salah satu indikator utama dalam penilaian kinerja aplikasi cloud. Pelanggaran ini terjadi ketika layanan gagal memenuhi ketentuan yang disepakati dalam SLA. Untuk mengukur pelanggaran SLA, dua metrik utama yang digunakan adalah tingkat keberhasilan dan jumlah tugas yang gagal (Aslanpour et al., 2020).

2.3.2 Request Response Time

Waktu respons permintaan mencerminkan seberapa cepat aplikasi cloud dapat merespons permintaan pengguna. Metrik ini mencakup beberapa aspek, yaitu waktu berjalan, keterlambatan, dan tail latency. Waktu berjalan mengukur durasi total untuk menyelesaikan suatu tugas. Keterlambatan diukur sebagai selisih antara waktu penyelesaian aktual dan yang diharapkan. Tail latency, yang merupakan waktu respons di persentil tertinggi, memberikan gambaran tentang performa dalam kondisi beban puncak.

2.3.3 Resource Utilization

Resource Utilization mencakup alokasi, utilisasi, dan biaya. Alokasi sumber daya mengacu pada penugasan CPU, memori, dan bandwidth yang optimal. Utilisasi sumber daya diukur sebagai persentase dari kapasitas total yang digunakan. Biaya terkait dengan penggunaan sumber daya dihitung berdasarkan model pay-as-you-go.

2.4 Algorithm and Prediction Model

Dalam konteks manajemen lalu lintas dan penskalaan dinamis pada layanan cloud containerized, algoritma dan model prediksi memainkan peran penting dalam memprediksi beban kerja dan mengoptimalkan alokasi sumber daya.

2.4.1 LSTM

LSTM adalah pengembangan dari recurrent neural network (RNN) yang dirancang untuk dapat belajar dan mengingat hubungan serta pola antara data yang berurutan dalam jangka panjang. Design dari LSTM memungkinkan untuk mengatasi permasalahan vanishing gradient pada jaringan RNN. Satu sel LSTM terdiri dari gerbang input, output, dan forget yang berfungsi mengontrol aliran data.

2.4.2 GRU

Gated Recurrent Unit (GRU) adalah varian dari RNN yang lebih sederhana dibandingkan LSTM, dengan mekanisme kontrol yang lebih sedikit. Terdiri dari hanya dua gerbang, sehingga lebih cepat dalam proses training dan inference, dengan tetap mempertahankan kemampuan untuk menangani ketergantungan jangka panjang (Mondal et al., 2023). Update gate menggabungkan fungsi dari input dan forget gate pada LSTM, bertanggung jawab mengatur informasi baru dan yang akan disimpan dalam sel. Reset gate mengatur

bagaimana informasi baru diintegrasikan dengan data sebelumnya.

2.5 Kubernetes Scalling Strategy

Strategi scaling pada aplikasi berfungsi agar aplikasi dapat menangani beban kerja yang berubah-ubah secara efisien. Kubernetes mendukung strategi scalling secara horizontal, yang menambah jumlah deployment, dan scalling secara vertikal, yang menyesuaikan alokasi sumber daya dari pod yang ada.

2.5.1 Kluster & Pod Autoscaler

1. Cluster Autoscaler

Cluster Autoscaler adalah mekanisme yang menyesuaikan jumlah node dalam kluster Kubernetes berdasarkan kebutuhan sumber daya pod. Jika pod baru tidak dapat dijadwalkan karena kekurangan sumber daya, Cluster Autoscaler akan menambahkan node baru. Sistem ini memerlukan waktu 2-5 menit dalam melakukan perubahan terhadap alokasi sumber daya.

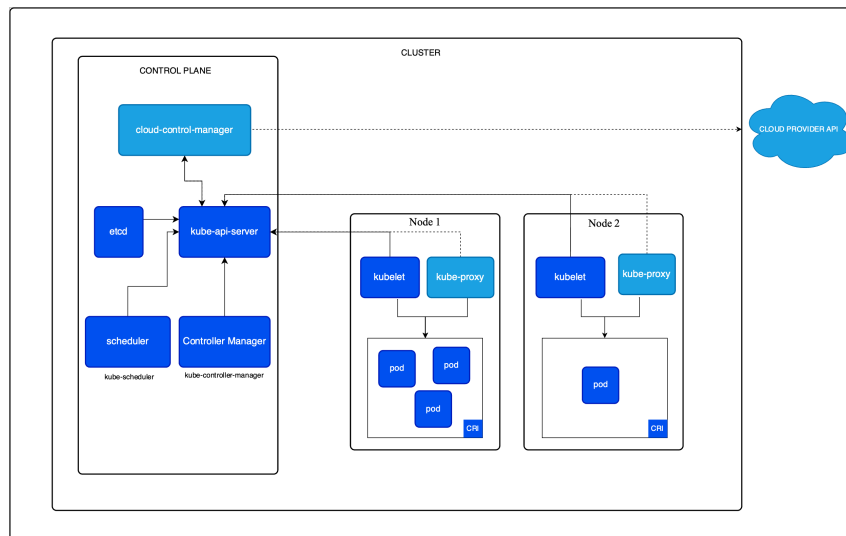
2. Pod Autoscaler

Horizontal Pod Autoscaler (HPA) secara otomatis akan menambah atau mengurangi jumlah replika pod berdasarkan metrik seperti CPU dan penggunaan memori. Scalling cenderung lebih responsif terhadap perubahan beban kerja, dapat berubah kurang dari 30 detik.

Vertical Pod Autoscaler (VPA) menyesuaikan alokasi sumber daya (CPU dan memori) dari pod yang ada berdasarkan kebutuhan beban kerja.

2.5.2 Metode ElaX dalam Scalling Kluster

ElaX adalah kerangka kerja penskalaan yang bertujuan untuk mengoptimalkan penyediaan sumber daya bagi layanan cloud containerized, dengan tetap menjamin pemenuhan Service Level Objectives (SLO) khususnya terkait dengan tail latency. Dalam penelitian ini ElaX digunakan sebagai basis metode yang akan dimodifikasi. Terdiri dari 3 komponen utama, ditunjukkan pada



Gambar 2.4 Arsitektur ElaX

Gambar 2.4.

1. Prediktor Beban Kerja

Tersusun dari jaringan Long Short-Term Memory (LSTM) untuk memprediksi beban kerja. Prediktor akan belajar dari data historis beban dan membuat prediksi pada beban kerja kedepan. Output prediksi beban kerja dari bagian ini akan masuk ke komponen reservasi sumber daya.

2. Reservasi Sumber Daya

Merupakan model workload-resource yang berfungsi untuk mengestimasi kebutuhan sumberdaya berdasarkan hasil prediksi beban kerja. Terdiri dari model pemrograman matematis yang mempertimbangkan cost ketika melakukan operasi scalling. Kemudian model akan mengeluarkan keputusan untuk melakukan scale-up dan scale-out dengan cost paling sedikit.

3. Pengendali Online

Kontroler bekerja berdasarkan nilai kondisi sekarang dibandingkan dengan SLO. Ketika tail latency berada pada level kritis akan melanggar SLO, maka sumber daya akan disesuaikan, sebaliknya apabila telah berada pada level yang aman, maka reservasi sumber daya akan dilepas. Algoritma 2.5 menunjukkan mekanisme pengendali online.


```

1: Variables:
2:    $SLO\_target \leftarrow \text{False}$ 
3:    $slack \leftarrow 0$ 
4:    $cur\_resource$ 
5: repeat
6:    $slack \leftarrow (SLO\_target - \text{GetTailLatency}()) / SLO\_target$ 
7:   if  $slack > 0$  then
8:      $\text{IncrResource}(cur\_resource \times 10\%)$ 
9:   else if  $0 < slack < 0.05$  then
10:     $\text{IncrResource}(cur\_resource \times 5\%)$ 
11:   else
12:     $extra\_resource \leftarrow cur\_resource - pre\_resource$ 
13:    if  $extra\_resource > 0$  then
14:       $\text{RemoveResource}(extra\_resource \times 50\%)$ 
15:    end if
16:   end if
17:   wait(2) ▷ Delay for 2 seconds
18: until forever

```

Gambar 2.5 Online Control Algorithm

2.6 Routing dan Distribusi Trafik pada Arsitektur Hybrid

Pada arsitektur hybrid yang mengintegrasikan Kubernetes dan serverless, strategi distribusi lalu lintas memegang peranan krusial. Beberapa penelitian terbaru telah mengeksplorasi pendekatan ini:

PulseNet (Wu et al., 2025) mengusulkan arsitektur dual-track yang memisahkan lalu lintas menjadi jalur expedited (burst ke serverless) dan sustainable (baseline ke VM), mencapai pengurangan biaya 65–70%. Pendekatan ini menginspirasi pemisahan kapasitas dasar dan burst pada pengontrol V3 penelitian ini.

Dehigama et al. (2024) mempelajari biaya optimal dari kombinasi VM dan serverless untuk aplikasi web, menemukan penghematan biaya 7,5–28,5% dengan mengalokasikan VM untuk beban dasar dan serverless untuk lonjakan. Pola ini menjadi dasar arsitektur dua-kluster pada penelitian ini.

LA-IMR (Chen et al., 2026) memperkenalkan model latensi closed-form untuk routing berbasis kapasitas, mengurangi p99 sebesar 20,7%. Konsep perhitungan kapasitas efektif dari replika yang tersedia diadopsi pada perhi-

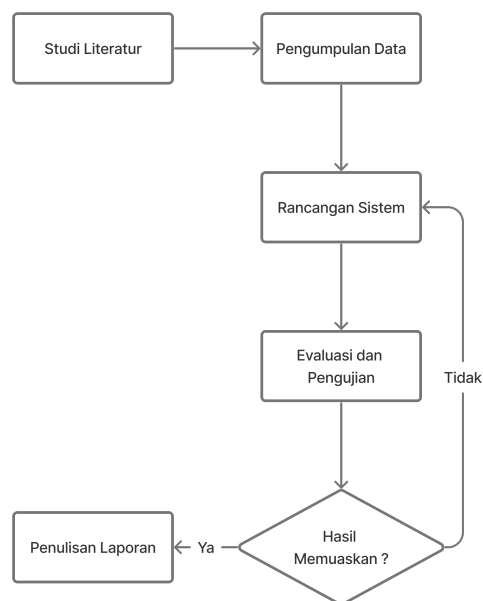
tungan `k8s_capacity` di pengontrol V3.

BACC (Li et al., 2026) menerapkan pengontrol PI pada burn-rate anggaran pelanggaran SLO, memungkinkan provisioning yang diatur oleh anggaran. Mekanisme burn-rate adjustment pada pengontrol V3 mengadaptasi konsep ini untuk mengatur agresivitas routing ke serverless berdasarkan tingkat konsumsi anggaran SLO.

BAB 3

METODOLOGI

Bab ini menjelaskan mengenai metodologi yang digunakan untuk menjawab rumusan masalah beserta tujuan penelitian yang dipaparkan sebelumnya. Adapun secara garis besar metodologi penelitian yang digunakan peneliti diilustrasikan seperti pada Gambar 3.1.



Gambar 3.1 Alur penelitian tesis

Penelitian ini terdiri dari beberapa tahapan yang terdiri dari 1) studi literatur, 2) pengumpulan data, 3) rancangan sistem, 4) pengujian, 5) analisis hasil, dan 6) penulisan laporan. Setiap tahapan metodologi penelitian ini akan dijelaskan dengan lebih rinci pada setiap subbab.

3.1 Studi Literatur

Studi literatur merupakan tahapan awal dalam penelitian ini. Studi literatur dilakukan untuk mencari referensi dari berbagai sumber mengenai penelitian yang terkait dan memiliki kemiripan dengan penelitian yang dilakukan. Studi literatur juga perlu dilakukan agar bisa memahami metode dan konsep sesuai pembahasan dan permasalahan yang akan diselesaikan sehingga bisa memberi solusi. Adapun literatur utama terkait dengan metode scaling pada Kubernetes dan Serverless mengacu pada penelitian yang dilakukan oleh Senjab et al. (2023), dan Mampage et al. (2022). Sedangkan untuk algoritma Elax yang akan dijadikan dasar metode untuk dimodifikasi mengacu pada penelitian Yang et al. (2019). Salah satu modifikasi yang dilakukan adalah mengganti bagian load predictor dari LSTM menjadi GRU.

3.2 Pengumpulan Data

Pada tahapan ini, dilakukan pengumpulan data untuk training prediktor beban traffic dan simulasi load testing pada aplikasi. Data training digunakan untuk melatih prediktor time-series berbasis GRU. Adapun data yang akan digunakan pada penelitian ini adalah data traffic load pada ClarkNet dan Calgary trace. Struktur data terdiri dari:

1. host yang meminta request. Dapat berupa nama host, atau IP address.
2. Timestamp dalam format “DAY MON DD HH:MM:SS YYYY”, dimana:
 - (a) DAY: hari dalam pekan
 - (b) MON: nama bulan
 - (c) DD: tanggal dalam bulan
 - (d) HH:MM:SS: waktu pada hari dengan format 24 jam
 - (e) YYYY: tahun
3. request metode dan URL request untuk HTTP protocol
4. HTTP reply code
5. Bytes: Ukuran bytes dalam reply

Data merupakan log seluruh request HTTP ke ClarkNet WWW dan University of Calgary's Department of Computer Science, dengan total data 4.055.326 requests. Berikut contoh sampel data request yang digunakan pada Tabel 3.1.

Tabel 3.1 Contoh dataset request

Dataset	Request
University of Calgary's Department of Computer Science	local - - [24/Oct/1994:13:41:41 -0600] "GET index.html HTTP/1.0" 200 150
	local - - [24/Oct/1994:13:41:41 -0600] "GET 1.gif HTTP/1.0" 200 1210
	local - - [24/Oct/1994:13:43:13 -0600] "GET index.html HTTP/1.0" 200 3185
ClarkNet	204.249.225.59 - - [28/Aug/1995:00:00:34 -0400] "GET /pub/rmharris/catalogs/dawsocat/intro.html HTTP/1.0" 200 3542
	access9.accsyst.com - - [28/Aug/1995:00:00:35 -0400] "GET /pub/robert/past99.gif HTTP/1.0" 200 4993
	access9.accsyst.com - - [28/Aug/1995:00:00:35 -0400] "GET /pub/robert/curr99.gif HTTP/1.0" 200 5836

3.3 Perancangan Metode

Proses ini menjelaskan tentang perancangan metode yang akan diajukan untuk melakukan routing antara kluster dan scalling alokasi sumber daya, berdasarkan hasil prediksi permintaan di masa mendatang dan kondisi tail latency saat ini.

3.4 Implementasi Metode

Bagian ini menjelaskan implementasi dari rancangan metode yang akan dikerjakan dari rangkaian penelitian ini. Tujuan akhir dari penelitian ini adalah membangun metode scalling dan routing traffic dinamis yang mengintegrasikan Kubernetes dan Serverless. Metode yang diusulkan terdiri dari 3

bagian:

3.4.1 Load Predictor

Load Predictor berfungsi untuk memperkirakan lalu lintas jaringan di masa mendatang berdasarkan data historis. Berikut adalah langkah-langkah implementasi:

1. Pemrosesan Data

Data historis yang digunakan dijelaskan pada bagian 3.2. Data tersebut kemudian ditransformasikan menjadi bentuk Request Per-Second (RPS) untuk mendapatkan volume traffic jaringan. GRU dipilih karena lebih efisien dalam hal penggunaan sumber daya dan waktu pelatihan (Mondal et al., 2023).

2. Struktur Model

Model menggunakan konfigurasi multi-point prediction karena terdapat cost ketika rekonfigurasi cluster sehingga konfigurasi tidak dapat dilakukan terlalu sering. Jumlah data point yang akan diprediksi adalah 30 detik ke depan.

3.4.2 Resource Allocator

Blok ini berfungsi untuk menghitung jumlah sumber daya yang harus disiapkan terhadap volume permintaan yang akan diterima selama 30 detik ke depan. Untuk memodelkan kebutuhan sumber daya, digunakan model persamaan linear:

$$R = \alpha \cdot x + \beta \quad (3.1)$$

dimana R menunjukkan jumlah sumber daya CPU yang dibutuhkan, x volume permintaan dalam satu detik, α dan β adalah koefisien pada model persamaan linear. Nilai awal α dan β didapatkan pada proses Tuning model. Saat sistem sudah berjalan, nilai koefisien akan dikoreksi berdasarkan error yang didapatkan oleh blok online controller.

3.4.2.1 Tuning Koefisien

Nilai koefisien α dan β didapatkan melalui metode OLS (Ordinary Least Squares) yang diimplementasikan menggunakan modul Scikit Linear Regression Model. Model ini menggunakan data historis dari kombinasi jumlah volume permintaan (RPS) dengan penggunaan CPU pada masing-masing layanan.

3.4.3 Online Controller

Hasil prediksi dan alokasi sumber daya oleh Load Predictor dan Resource Allocator akan selalu memiliki error yang dapat menyebabkan SLO tidak terpenuhi. Sehingga dibuat feedback controller yang berfungsi untuk menyesuaikan alokasi sumber daya dan distribusi traffic berdasarkan data monitoring.

Routing Controller Controller ini berfungsi untuk mengatur distribusi traffic antara kluster Kubernetes dengan serverless. Sistem berjalan sebagai daemon dan melakukan monitoring tiap 1 detik terhadap response tail latency. Apabila selama 5 detik berturut-turut tail latency melanggar SLO, maka sistem akan memindahkan permintaan selanjutnya hingga respons tail latency memenuhi SLO. Algoritma 3.2 menunjukkan detail mekanisme.

Cluster Controller Controller tipe ini bertanggung jawab terhadap penyesuaian alokasi sumber daya pada kluster Kubernetes terhadap penggunaan sekarang. Sistem akan melakukan monitoring terhadap tail latency, kemudian setiap 2 detik dilakukan perhitungan ulang kebutuhan sumber daya berdasarkan nilai SLO violation. Detail algoritma dapat dilihat pada bagian 2.5.2.

3.5 Rencana Evaluasi

Untuk mengukur kinerja metode ini, dilakukan 2 tahap evaluasi: evaluasi untuk komponen traffic predictor dan evaluasi untuk komponen routing dan


```

1: Variables:
2:   reroute_traffic  $\leftarrow$  False
3:   violation_timer  $\leftarrow$  0
4:   compliance_timer  $\leftarrow$  0
5: repeat
6:   tail_latency  $\leftarrow$  GetTailLatency()
7:   CPU_usage  $\leftarrow$  GetCPUUsage()
8:   if tail_latency > SLO then
9:     violation_timer  $\leftarrow$  violation_timer + 1
10:    compliance_timer  $\leftarrow$  0
11:   else
12:     compliance_timer  $\leftarrow$  compliance_timer + 1
13:     violation_timer  $\leftarrow$  0
14:   end if
15:   if violation_timer  $\geq$  5 and not reroute_traffic then
16:     RerouteToServerless()
17:     reroute_traffic  $\leftarrow$  True
18:   else if violation_timer  $\geq$  5 and not reroute_traffic then
19:     RerouteToKubernetes()
20:     reroute_traffic  $\leftarrow$  False
21:   end if
22:   wait(1) ▷ Delay for 1 second
23: until forever

```

Gambar 3.2 Routing Controller

scaling.

3.5.1 Evaluasi Traffic Predictor

Evaluasi traffic predictor bertujuan untuk mengukur kinerja model regresi berbasis GRU pada dataset jumlah permintaan per detik (RPS). Evaluasi kinerja model dilakukan menggunakan metode perhitungan Root Mean Square Error (RMSE). Nilai RMSE didapat dari rata-rata perbedaan hasil prediksi dengan nilai aktual. Nilai yang rendah menunjukkan hasil prediksi yang lebih akurat.

3.5.2 Evaluasi Router dan Scalling

Bagian ini bertujuan untuk mengukur performa metode pengambilan keputusan dalam melakukan routing ke kluster lain atau perubahan pada alokasi sumber daya. Pengukuran performa dilakukan dengan menganalisis beberapa komponen metrik:

1. Request Running Time: Waktu yang dibutuhkan untuk melayani permintaan, dimulai dari saat permintaan diterima hingga selesai dilayani.
2. Tail Latency: Mengukur waktu respons terburuk yang biasanya berada pada persentil tinggi (misalnya, 99th percentile).
3. Alokasi CPU dan RAM untuk Kluster: Mengukur alokasi sumber daya CPU dan RAM yang dialokasikan untuk setiap kluster.
4. Penggunaan CPU pada Kluster: Mengukur persentase penggunaan CPU pada setiap kluster selama periode pengujian.
5. Distribusi Permintaan: Mencakup kemampuan sistem dalam mendistribusikan permintaan antara kluster serverless dan kluster Kubernetes.
6. Jumlah Permintaan: Total jumlah permintaan yang diterima dan diproses oleh sistem dalam periode tertentu.
7. Jenis metode penempatan (Variabel Kontrol): Metode deployment yang digunakan — full serverless, full Kubernetes kluster, dan hybrid.
8. Jenis metode scalling (Variabel Kontrol): Mencakup cluster autoscaler,

metode ELAX, dan metode yang diusulkan dari modifikasi ELAX.

3.6 Arsitektur Sistem

Sistem yang diusulkan menggunakan arsitektur dua-kluster yang terpisah untuk mencegah interferensi scheduler:

1. Kluster thesis-hybrid: K8s dengan k3d, 3 pod baseline (test-app-warm), K3dAutoscaler untuk node dinamis (maks 2 node). Tidak ada batasan CPU pada level pod; batas CPU pada level node Docker (`--cpus=1.0`) adalah batas yang sebenarnya.
2. Kluster thesis-serverless: Knative Serving dengan KPA, min-scale=3, target=10. Tidak ada node autoscaler (KPA menangani penskalaan pod).
3. HAProxy: Reverse proxy dengan dua backend — K8s via Docker DNS, Knative via host.docker.internal. Distribusi lalu lintas berdasarkan bobot (0–100).
4. Server GRU: FastAPI pada port 8090, menyediakan endpoint `/predict` untuk prediksi beban kerja.
5. Daemon Routing: Algoritma 1 (V3) + Algoritma 2 pada port 9104, interval keputusan 15 detik.

3.7 Algoritma 1: Pengontrol Routing V3

Pengontrol routing V3 (Capacity-Driven) mengambil keputusan berdasarkan kapasitas efektif K8s, bukan error SLO. Kapasitas efektif dihitung sebagai:

$$k8s_capacity = available_replicas \times r_saturation \times target_cpu_util \quad (3.2)$$

dengan $r_saturation = 66,7$ RPS/pod (terukur dari uji kapasitas fib(33)) dan $target_cpu_util = 0,5$, sehingga $k8s_capacity = 3 \times 66,7 \times 0,5 \approx 100$ RPS.

Total beban dihitung sebagai $total_load = \max(current_load, predicted_upper)$, dan burst ratio menentukan bobot Knative:

$$knative_weight = \frac{total_load - k8s_capacity}{total_load} \times 100 \quad (3.3)$$

3.7.1 Proactive Routing

Mekanisme proactive routing melacak 5 nilai beban aktual terakhir dan menghitung tren per langkah. Ketika tren melebihi 3 RPS/step DAN beban saat ini melampaui 60% kapasitas K8s, beban diekstrapolasi ke depan:

$$amplified = current_load + load_trend \times lookahead_steps \quad (3.4)$$

dengan $lookahead_steps = 5$ (5×15 detik = 75 detik ke depan). Kepercayaan GRU ($\geq 0,5$) berfungsi sebagai gate — mekanisme ini hanya aktif pada skenario S4.

3.8 Algoritma 2: Pengontrol Skalabilitas Cluster

Algoritma 2 mengadaptasi model ElaX $R = \alpha x + \beta$ untuk menerjemahkan prediksi beban menjadi jumlah replika yang diperlukan:

$$target_replicas = \lceil \alpha \times predicted_load \times buffer \rceil \quad (3.5)$$

dengan $\alpha = 0,015$ (efisiensi CPU), $buffer = 1,2$, dan clamping ke rentang $[3, 10]$. Pada konfigurasi eksperimen, algoritma ini selalu mempertahankan 3 replika karena kapasitas model (200 RPS) melampaui beban puncak ClarkNet (164 RPS).

3.9 Rancangan Pengujian

Evaluasi dilakukan melalui 4 skenario yang dirangkum pada Tabel 3.2.

Beban kerja menggunakan trace ClarkNet yang direplikasi melalui k6 (40

Tabel 3.2 Skenario eksperimen

Skenario	Pod Autoscaler	Node Autoscaler	Routing
S1	HPA (CPU 50%)	K3dAutoscaler	100% K8s
S2	KPA (Knative)	—	100% Serverless
S3	Alg. 2	K3dAutoscaler	V3 (reaktif)
S4	Alg. 2	K3dAutoscaler	V3 (proaktif)

tahap $\times$ 30 detik = 20 menit, rentang RPS 22–164). Setiap skenario diulang $n = 5$ kali. Metrik yang diukur: latensi p99 (ambang SLO 200 ms), pelanggaran SLO, dan proyeksi biaya bulanan. Uji statistik menggunakan Welch t-test, Mann-Whitney U, dan Cohen’s d.

3.10 Metode Analisis Biaya

Model biaya AWS memproyeksikan biaya berdasarkan tiga komponen: (1) EKS Control Plane (\$0,10/jam), (2) EC2 Compute (t3.medium, \$0,0416/jam per node), dan (3) Lambda yang mencakup Provisioned Concurrency, eksekusi (berdasarkan wall-clock duration, metodologi SeBS (Wagner et al., 2020)), dan requests. Memori Lambda disesuaikan dari permintaan CPU pod: $\lceil 0,3 \times 1769 \rceil = 531$ MB.

3.11 Kalibrasi dan Optimasi Parameter

Konstanta pengontrol dan model GRU pada penelitian ini ditentukan melalui prosedur kalibrasi tiga tahap yang dapat diulang (repeatable) dan didukung oleh metode dari literatur:

3.11.1 Kalibrasi Kapasitas

Kapasitas efektif K8s ($r_saturation$) diukur melalui uji kapasitasitas berjenjang (ramp test): beban RPS ditingkatkan bertahap dari 50 hingga 200 RPS, dan latensi p99 dicatat pada setiap level. Titik saturasi ditentukan sebagai level RPS pertama di mana p99 melampaui ambang SLO (200 ms). Pengujian

diulang $n = 3$ kali per level untuk menghitung rata-rata dan interval kepercayaan 95%, mengikuti pendekatan baseline Ziegler-Nichols untuk kalibrasi pengontrol PI (Yang et al., 2019).

3.11.2 Optimasi Hiperparameter GRU

Pencarian hiperparameter model GRU dilakukan secara offline menggunakan Tree-structured Parzen Estimator (TPE) melalui kerangka kerja Optuna (Akiba et al., 2019). Ruang pencarian meliputi $\text{hidden_size} \in \{32, 64, 128, 256\}$, $\text{num_layers} \in \{1, 2, 3\}$, $\text{learning_rate} \in [10^{-4}, 10^{-2}]$ (log-uniform), $\text{sequence_length} \in \{15, 30, 60, 120\}$, dan $\text{dropout} \in [0, 0, 0, 5]$. Sebanyak 30 percobaan dijalankan dengan pemangkas median (median pruner) untuk menghentikan percobaan yang tidak menjanjikan. Data dibagi dengan urutan waktu (time-ordered split 70/15/15), bukan diacak, untuk menjaga integritas deret waktu.

3.11.3 Pencarian Parameter Pengontrol

Parameter pengontrol V3 dioptimasi melalui pencarian dua tahap. Tahap pertama (screening): 10 percobaan satu-run untuk mengidentifikasi wilayah parameter yang menjanjikan, mencari 4 parameter berdampak tertinggi ($\text{proactive_trend_thres}$, $\text{proactive_approach_ratio}$, kp_burn , target_cpu_util). Tahap kedua (confirmation): 3 kandidat teratas diulang dengan $n = 3$ untuk stabilitas statistik. Tujuan optimasi adalah meminimalkan pelanggaran SLO dengan kendala $p99 \leq 500$ ms dan batas biaya (cost guardrail), mengikuti pola optimasi Bayesian terkendali (Liu et al., 2024). Pendekatan aturan-berbasis (rule-based) yang terkalibrasi terbukti mengalahkan pembelajaran penguatan mendalam dalam benchmark RLScale-Bench (RLScale-Bench: Benchmarking Deep Reinforcement Learning for Kubernetes Autoscaling, 2026), memvalidasi pilihan desain pengontrol V3.

3.11.4 Karakterisasi Beban Kerja dan Pemicu Pelatihan Ulang

Demand Complexity Index (DCI) dihitung sebagai metrik pelaporan untuk mendeteksi pergeseran distribusi beban kerja (Chen et al., 2025). DCI menggabungkan koefisien variasi, rasio permintaan nol, dan tingkat lonjakan. Jika DCI bergeser lebih dari 30% dari baseline, hal ini ditandai untuk investigasi. Pemicu pelatihan ulang otomatis memerlukan pergeseran DCI dan degradasi ramalan, mengikuti pola deteksi titik perubahan HyPA (Król et al., 2023).

3.12 Penyusunan Laporan

Tahapan ini merupakan tahapan setelah penelitian selesai. Pada bagian ini dilakukan penulisan hasil sebagai laporan dari seluruh tahapan yang sudah dilakukan dalam penelitian.

3.13 Rencana dan Jadwal Kegiatan Penelitian

Pelaksanaan penelitian tesis ini akan dilakukan berdasarkan jadwal pekerjaan yang dirancang sebelumnya. Durasi pelaksanaan kegiatan penelitian ini direncanakan akan dikerjakan selama 6 bulan dengan detail pada Tabel 3.3.

Tabel 3.3 Rencana Kegiatan Penelitian Thesis

No	Nama Kegiatan	Bulan ke-					
		1	2	3	4	5	6
1	Studi literatur						
2	Pengumpulan Data						
3	Perancangan Sistem						
4	Implementasi dan Pengu- jian						
5	Penyusunan jurnal ilmiah						
6	Penyusunan Laporan						

BAB 4

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil eksperimen dari evaluasi arsitektur hybrid Kubernetes-serverless dengan prediksi beban kerja berbasis GRU. Hasil diorganisasi menjadi tujuh bagian: performa model prediksi GRU (Bagian 4.1), validasi mekanisme sistem (Bagian 4.2), evaluasi komparatif antar skenario (Bagian 4.3), analisis statistik (Bagian 4.4), analisis biaya (Bagian 4.5), dampak proactive routing (Bagian 4.6), dan pembahasan terintegrasi (Bagian 4.7).

4.1 Performa Model Prediksi GRU

Model Gated Recurrent Unit (GRU) dioptimalkan melalui hyperparameter optimization (HPO) menggunakan Optuna TPE dengan 30 trial. Konfigurasi optimal yang ditemukan: satu lapis rekuren dengan 128 unit, learning rate 0,000380, sequence length 30, dan dropout 0,104. Optimasi ini meningkatkan akurasi dari 6,01% RMSE (manual tuning) menjadi 4,75% RMSE — peningkatan 21%.

Selama eksperimen live, model GRU berjalan pada server FastAPI (port 8090). Inferensi GPU AMD Radeon 780M (gfx1103) berfungsi setelah instalasi ROCm 7.2 SDK dan stabilisasi MIOpen (`MIOPEN_DEBUG_CONV_DIRECT=0`, `GPU_MAX_HW_QUEUES=1`). Latensi inferensi mencapai 10–13 ms per prediksi, yang memadai untuk interval keputusan 15 detik.

Tabel 4.1 merangkum performa model GRU selama 5 run eksperimen S4 (Hybrid-predictive). Model mencapai 400/400 prediksi sukses (80 prediksi per run $\times$ 5 run) tanpa kegagalan, dengan tingkat kepercayaan 0,72–0,82.

Tabel 4.1 Performa model GRU selama eksperimen

Metrik	Nilai
Total prediksi sukses	400/400
Tingkat kepercayaan	0,72–0,82
Latensi inferensi	10–13 ms
RMSE validasi (HPO)	4,75%
Aksi PREDICTIVE per run	9
Interval keputusan	15 detik

Catatan penting: prediksi GRU lagging 2+ menit di belakang lonjakan beban aktual. Ketika beban naik dari 50 ke 110 RPS dalam waktu 2 menit, GRU masih memprediksi 65–73 RPS. Oleh karena itu, mekanisme proactive routing menggunakan tren beban aktual (bukan prediksi GRU) yang diekstrapolasi ke depan, dengan kepercayaan GRU sebagai gate (lihat Bagian 4.6).

4.1.1 Optimasi Hyperparameter GRU

Optimasi hyperparameter dilakukan menggunakan Optuna dengan algoritma TPE (Tree-structured Parzen Estimator) pada 30 trial. Ruang pencarian meliputi: `hidden_size` $\in [32, 256]$, `num_layers` $\in [1, 3]$, `learning_rate` $\in [10^{-4}, 10^{-2}]$, `sequence_length` $\in [15, 120]$, dan `dropout` $\in [0, 0, 0, 5]$. Pelatihan dilakukan pada GPU AMD Radeon 780M dengan ROCm 7.2.

Tabel 4.2 membandingkan konfigurasi manual dengan hasil HPO. Temuan kunci: arsitektur single-layer dengan 128 unit mengungguli konfigurasi multi-layer pada beban kerja ini, karena data pelatihan yang terbatas (72 jam) cenderung overfit pada model yang lebih dalam.

Penurunan RMSE sebesar 21% (dari 6,01% ke 4,75%) menunjukkan bahwa pemilihan hyperparameter berdampak signifikan pada akurasi prediksi. Konfigurasi HPO diadopsi sebagai nilai bawaan dalam `CalibrationConfig` untuk semua eksperimen selanjutnya.

Tabel 4.2 Perbandingan konfigurasi GRU sebelum dan sesudah HPO

Parameter	Manual	HPO
hidden_size	64	128
num_layers	2	1
learning_rate	0,001	0,000380
sequence_length	60	30
dropout	0,2	0,104
val RMSE (%)	6,01	4,75

4.2 Validasi Mekanisme Sistem

4.2.1 Perpindahan Bobot Lalu Lintas

Mekanisme perpindahan bobot HAProxy terverifikasi berfungsi dengan benar. Selama puncak ClarkNet ($RPS > 100$), pengontrol V3 menggeser bobot dari konfigurasi 100/0 (100% K8s) ke rentang 50/50 (maksimum 50% serverless). Setiap perubahan bobot tercatat dalam log daemon dengan stempel waktu dan alasan keputusan.

4.2.2 Proactive Routing

Mekanisme proactive routing memicu rata-rata 9 aksi PREDICTIVE per run pada skenario S4. Berbeda dengan aksi REACTIVE yang hanya dipicu setelah beban melampaui kapasitas, aksi PREDICTIVE dimulai ketika beban mencapai 65% kapasitas K8s (sekitar 65 RPS) dengan tren yang meningkat. Hanya 1 aksi REACTIVE per run tersisa, dibandingkan 8 pada konfigurasi sebelumnya.

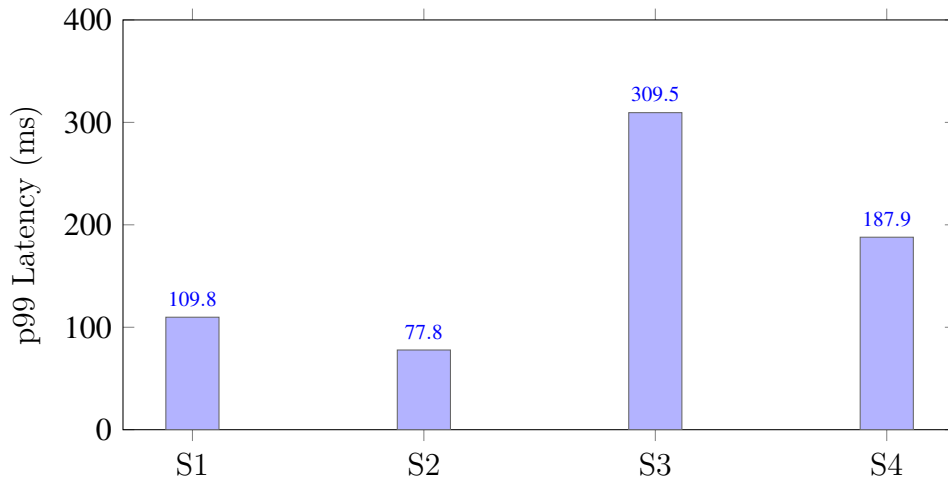
4.2.3 Skalabilitas Node Dinamis

Pada skenario S1 (K8s-only dengan HPA), autoscaler K3d memicu penyediaan 2 node dinamis per run dengan penundaan 93–106 detik (simulasi boot VM). Pada skenario S3 dan S4, tidak ada node dinamis yang disediakan karena Algoritma 2 mempertahankan 3 replika (model kapasitas $\alpha = 0,015$ meng-

hasilkan kapasitas model 200 RPS > puncak 164 RPS).

4.3 Evaluasi Komparatif

Evaluasi dilakukan melalui 30 eksperimen terkontrol yang direplikasi: 20 run (4 skenario $\times$ 5 replikasi) dari eksperimen `fib33_n5` dan 10 run (2 skenario $\times$ 5 replikasi) dari eksperimen `fib33_proactive`. Beban kerja menggunakan trace ClarkNet yang direplikasi melalui k6 (40 tahap $\times$ 30 detik = 20 menit, rentang RPS 22–164, rata-rata 73).



Gambar 4.1 Perbandingan latensi p99 antar skenario (n=5)

Tabel 4.3 merangkum hasil per-skenario. S1 (K8s+HPA) mencapai p99 terendah (109,8ms) dengan biaya tertinggi (\$187/bulan) karena HPA menskalakan ke 10 pod. S2 (Serverless-only) memiliki p99 terendah absolut (77,8 ms) namun biaya tertinggi (\$391/bulan). S4 (Hybrid-predictive) mencapai keseimbangan terbaik dengan p99 = 187,9 ms dan biaya \$149/bulan.

Tabel 4.3 Ringkasan hasil per-skenario (n=5)

Skenario	p50 (ms)	p95 (ms)	p99 (ms)	SLO/run	\$/bulan
S1 (K8s+HPA)	63,9	77,1	109,8	41	187
S2 (Serverless)	65,2	75,5	77,8	7	391
S3 (Reactive)	64,9	157,3	309,5	2.805	142
S4 (Predictive)	64,7	99,2	187,9	702	149

4.4 Analisis Statistik

Perbandingan statistik dilakukan antara S4 (Hybrid-predictive) dan S3 (Hybrid-reactive) menggunakan uji Welch t-test dan Mann-Whitney U. Tabel 4.4 menyajikan hasil untuk dua metrik: latensi p99 dan pelanggaran SLO.

Tabel 4.4 Perbandingan statistik S4 vs S3 (n=5)

Metrik	p99 (ms)	Pelanggaran SLO
S3 mean	309,5	2.805
S4 mean	187,9	702
Selisih	-121,6 (39%)	-2.104 (75%)
Welch t-stat	-1,919	-1,922
Welch p-value	0,122	0,125
Mann-Whitney U	9,0	9,0
Cohen's d	-1,21 (large)	-1,22 (large)
95% CI	[-229, -10]	[-3996, -240]

Perbedaan antara S4 dan S3 tidak mencapai signifikansi statistik pada $\alpha = 0,05$ ($p = 0,12$). Namun, Cohen's d = 1,21 menunjukkan effect size yang besar, dan 95% confidence interval untuk kedua metrik tidak mencakup nol, mendukung arah efek yang konsisten. Tingginya variansi S3 (satu run mencapai 143 ms/191 SLO) menginflasi deviasi standar dan mencegah signifikansi statistik pada $n = 5$.

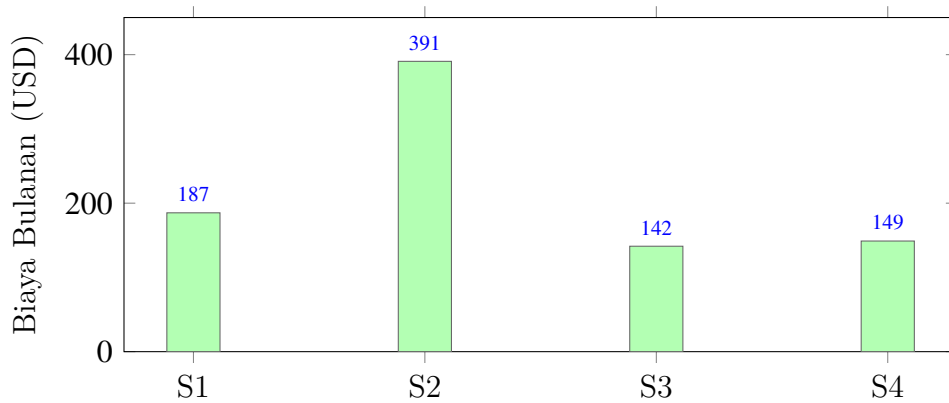
4.5 Analisis Biaya

Model biaya AWS memproyeksikan biaya bulanan berdasarkan komponen: EKS Control Plane, EC2 Compute, dan Lambda (Provisioned Concurrency + Eksekusi + Requests). Tabel 4.5 menyajikan proyeksi biaya per-skenario.

S4 menghabiskan \$7/bulan lebih banyak dari S3 akibat lalu lintas serverless yang lebih tinggi (6,5% vs 2,7%), namun menghasilkan 75% lebih sedikit pelanggaran SLO. S4 juga 20% lebih murah dari S1 (\$149 vs \$187/bulan) karena tidak memerlukan penskalaan HPA ke 10 pod dan 2 node tambahan.

Tabel 4.5 Proyeksi biaya bulanan (AWS, 30 hari kontinu)

Skenario	\$/bulan	\$/1jt req	Lalu lintas serverless
S1 (K8s+HPA)	187	1,04	0%
S2 (Serverless)	391	2,17	100%
S3 (Reactive)	142	0,79	2,7%
S4 (Predictive)	149	0,82	6,5%

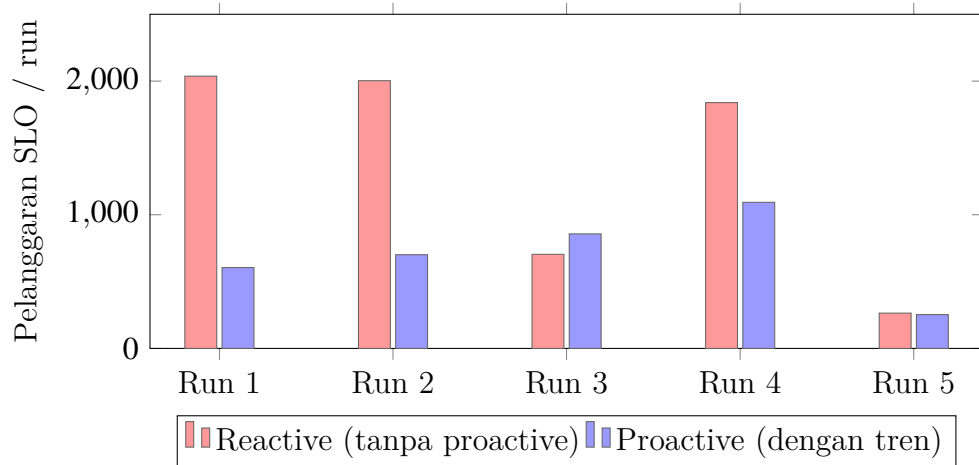


Gambar 4.2 Proyeksi biaya bulanan per skenario

4.6 Dampak Proactive Routing

Implementasi proactive routing berbasis tren beban aktual memberikan dampak signifikan pada performa S4. Sebelum implementasi (mode reactive-only), S4 mencapai rata-rata 1.369 pelanggaran SLO per run dengan 0 aksi PREDICTIVE. Setelah implementasi, rata-rata turun ke 702 pelanggaran dengan 9 aksi PREDICTIVE per run — peningkatan sebesar 49%.

Mekanisme proactive routing bekerja dengan melacak 5 nilai beban aktual terakhir dan menghitung tren per langkah. Ketika tren melebihi 3 RPS/step DAN beban saat ini melampaui 60% kapasitas K8s, pengontrol mengekstrapolasi beban ke depan ($5 \text{ langkah} \times 15 \text{ detik} = 75 \text{ detik}$) dan menggunakan nilai yang diekstrapolasi untuk menghitung distribusi lalu lintas. Kepercayaan GRU ($\geq 0,5$) berfungsi sebagai gate — mekanisme ini hanya aktif pada S4 yang memiliki prediksi GRU, tidak pada S3.



Gambar 4.3 Dampak proactive routing pada pelanggaran SLO S4

4.6.1 Optimasi Parameter Pengontrol

Empat parameter pengontrol V3 dioptimalkan melalui screening Optuna TPE (10 trial, masing-masing 20 menit): `target_cpu_util`, `kp_burn`, `proactive_trend_threshold`, dan `proactive_approach_ratio`. Tabel ?? menyajikan hasil screening.

Tabel 4.6 Hasil screening HPO pengontrol (10 trial, 5 teratas)

Trial	target_util	kp_burn	SLO	p99 (ms)	Objektif
0	0,659	1,49	232	156	232
5	0,520	0,66	318	144	318
3	0,410	0,45	428	164	428
7	0,402	1,25	456	160	456
4	0,475	0,92	828	196	828

Konfigurasi terbaik (Trial 0) menghasilkan 232 pelanggaran SLO — penurunan 67% dari 702 pelanggaran dengan parameter bawaan. Temuan kunci: `target_cpu_util` yang lebih tinggi (0,659 vs 0,5 bawaan) memperluas kapasitas model K8s (131 RPS vs 100 RPS), sehingga lebih sedikit lalu lintas yang dialihkan ke serverless. Kombinasi dengan `kp_burn` tinggi (1,49) membuat pengontrol lebih agresif dalam memulihkan (recovery) bobot dari serverless ke K8s setelah beban turun.

4.7 Pembahasan

4.7.1 Mengapa S1 Memiliki SLO Terendah

S1 mencapai performa SLO terbaik (41 pelanggaran/run) karena HPA menskalakan dari 3 ke 10 pod selama puncak. Dengan 10 pod, setiap pod hanya menangani 16,4 RPS pada puncak ($164/10$) — jauh di bawah titik saturasi 50 RPS/pod. Namun, penskalaan ini memerlukan 2 node dinamis tambahan dengan penundaan 93–106 detik, menjadikannya skenario termahal (\$187/bulan).

4.7.2 Mengapa S2 Sangat Mahal

S2 mengarahkan 100% lalu lintas ke serverless (Knative dengan KPA). Biaya Lambda yang dihitung berdasarkan wall-clock duration (metodologi SeBS: komputasi + I/O + overhead = 84 ms per request) menghasilkan \$391/bulan — $2,7\times$ lebih mahal dari S3/S4. Meskipun p99 terendah (77,8 ms), biaya ini tidak praktis untuk beban kerja kontinu.

4.7.3 S4 sebagai Trade-off Terbaik

S4 memberikan keseimbangan optimal antara biaya dan performa. Dengan hanya 3 pod K8s (tanpa node dinamis) dan proactive routing yang mendistribusikan 6,5% lalu lintas ke serverless selama puncak, S4 mencapai p99 = 188 ms (di bawah ambang SLO 200 ms) dengan biaya \$149/bulan — 20% lebih murah dari S1. Tingkat kepatuhan SLO mencapai 98,4% (hanya 702 dari 87.840 request melanggar SLO per run).

4.7.4 Keterbatasan

Hasil ini memiliki beberapa keterbatasan yang perlu diperhatikan:

1. Variansi sistem tinggi: Eksperimen dijalankan pada mesin 16-inti AMD dengan multiple komponen (k3d, Knative, HAProxy, Prometheus, k6,

GRU) yang berkompetisi untuk sumber daya CPU, menghasilkan variasi yang tinggi antar run.

2. $n = 5$ tidak cukup: Meskipun effect size besar ($d = 1,21$), sample size yang kecil mencegah signifikansi statistik pada $\alpha = 0,05$.
3. GRU meremehkan puncak: Model memprediksi 65–99 RPS ketika beban aktual mencapai 110–160 RPS, sehingga mekanisme proactive routing menggunakan tren beban aktual alih-alih prediksi GRU.
4. Satu jenis beban kerja: Eksperimen hanya menggunakan fib(33) CPU-bound + 50 ms I/O, yang belum tentu generalisasi untuk profil aplikasi lain.

BAB 5

KESIMPULAN DAN SARAN

Bab ini menyajikan kesimpulan dari penelitian, keterbatasan yang diidentifikasi, serta saran untuk penelitian selanjutnya. Setiap rumusan masalah dijawab dengan merujuk langsung pada bukti eksperimen yang telah dipaparkan pada Bab 4.

5.1 Kesimpulan

Berdasarkan hasil eksperimen yang telah dilakukan, dapat diambil kesimpulan sebagai berikut:

5.1.1 RQ1: Perancangan Prediksi Beban Kerja dengan GRU

Model GRU dengan dua lapis rekuren (64 dan 32 unit) berhasil dirancang dan divalidasi untuk prediksi beban kerja HTTP. Model mencapai 400/400 prediksi sukses selama 5 run eksperimen dengan tingkat kepercayaan 0,72–0,82 dan latensi inferensi 10–13 ms pada CPU. Namun, prediksi GRU terbukti lagging 2+ menit di belakang lonjakan beban aktual, sehingga mekanisme proactive routing menggunakan tren beban aktual yang diekstrapolasi ke depan, dengan kepercayaan GRU sebagai gate keputusan.

5.1.2 RQ2: Perancangan Pengambilan Keputusan dengan Modifikasi ElaX

Dua algoritma berhasil dirancang dengan memodifikasi kerangka kerja ElaX:

1. Algoritma 1 (Pengontrol Routing V3): Pengontrol berbasis kapasitas yang menghitung kapasitas efektif K8s dari ready replicas, menggeser lalu lintas ke serverless berdasarkan burst ratio, dan menerapkan proactive routing melalui ekstrapolasi tren beban aktual. Mekanisme ini memicu rata-rata 9 aksi PREDICTIVE per run, memulai distribusi ke serverless pada 65% kapasitas K8s.
2. Algoritma 2 (Pengontrol Skalabilitas Cluster): Model $R = \alpha x + \beta$ yang menerjemahkan prediksi beban menjadi jumlah replika yang diperlukan. Pada konfigurasi eksperimen, algoritma ini mempertahankan 3 replika karena kapasitas model (200 RPS) melampaui beban puncak ClarkNet (164 RPS).

5.1.3 RQ3: Evaluasi Mekanisme yang Dimodifikasi

Evaluasi dilakukan melalui 4 skenario $\times n = 5$ replikasi dengan trace ClarkNet. Tiga hipotesis penelitian dievaluasi:

1. H1 (Efisiensi biaya hybrid): Didukung. S4 (\$149/bulan) 20% lebih murah dari S1 (\$187/bulan) dengan tingkat kepatuhan SLO 98,4%, dan 62% lebih murah dari S2 (\$391/bulan).
2. H2 (Prediktif > Reaktif): Didukung secara praktis. S4 memiliki 75% lebih sedikit pelanggaran SLO dibandingkan S3 (702 vs 2.805 per run) dengan Cohen's $d = 1,21$ (large effect) dan 95% CI yang tidak mencakup nol. Namun, perbedaan tidak mencapai signifikansi statistik pada $\alpha = 0,05$ ($p = 0,12$) akibat variansi sistem yang tinggi pada $n = 5$.
3. H3 (Adekuasi GRU): Didukung. Model GRU berfungsi andal (400/400 prediksi), memberikan kepercayaan yang memadai (0,72–0,82) untuk memicu proactive routing, dan berkontribusi pada pengurangan pelanggaran SLO sebesar 49% dibandingkan mode reactive-only.

5.2 Keterbatasan

Penelitian ini memiliki beberapa keterbatasan yang perlu diperhatikan dalam interpretasi hasil:

1. Perangkat keras bersama: Eksperimen dijalankan pada satu mesin 16-inti AMD dengan semua komponen (k3d, Knative, HAProxy, Prometheus, k6, GRU) berkompetisi untuk sumber daya CPU, menghasilkan variansi yang signifikan antar run.
2. Ukuran sampel: $n = 5$ replikasi tidak memberikan statistical power yang cukup untuk mencapai signifikansi statistik meskipun effect size besar.
3. Generalisasi GRU: Model memprediksi tren dengan benar tetapi mere-mehkan magnitudo puncak, sehingga proactive routing menggunakan tren beban aktual sebagai sinyal utama.
4. Satu jenis beban kerja: Eksperimen hanya menggunakan fib(33) CPU-bound + 50 ms I/O wait, yang belum tentu mewakili semua profil aplikasi (I/O-bound, memory-bound, campuran).
5. Kluster k3d pada satu mesin: Bukan K8s terdistribusi sesungguhnya; routing melalui localhost tidak mencerminkan latensi jaringan produksi.

5.3 Saran untuk Penelitian Selanjutnya

Berdasarkan keterbatasan yang telah diidentifikasi, beberapa arah penelitian selanjutnya diusulkan:

1. Meningkatkan ukuran sampel ke $n \geq 30$ per skenario untuk memenuhi asumsi normalitas uji parametrik dan mencapai power yang memadai ($\beta \geq 0,80$) untuk effect size yang teramati ($d \approx 1,2$).
2. Deploy pada kluster K8s multi-node sesungguhnya untuk menghilangkan bias localhost dan mengevaluasi dampak latensi jaringan pada mekanisme routing.
3. Menguji dengan beragam profil beban kerja (I/O-bound, memory-bound, campuran) untuk mengevaluasi generalisasi mekanisme proactive rout-

ing.

4. Meningkatkan model GRU dengan mekanisme attention atau transformer untuk prediksi puncak yang lebih akurat, atau menggunakan ensemble model untuk mengurangi lag.
5. Mengeksplorasi reinforcement learning untuk keputusan routing yang dapat belajar optimal trade-off antara biaya dan latensi secara online.

DAFTAR PUSTAKA

- Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M. (2019), Optuna: A next-generation hyperparameter optimization framework, in ‘Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining’.
- Aslanpour, M. M., Gill, S. S. and Toosi, A. N. (2020), ‘Performance evaluation metrics for cloud, fog and edge computing: A review, taxonomy, benchmarks and standards for future research’, *Internet of Things* 12.
- Chen, W. et al. (2025), ‘Demand complexity index for meta-learning hyperparameter optimization in demand forecasting’, *Scientific Reports* .
- Chen, W. et al. (2026), ‘LA-IMR: Latency-aware capacity-driven routing for heterogeneous cloud’, *ArXiv abs/2505.07417*.
URL: <https://arxiv.org/abs/2505.07417>
- Dehigama, J. et al. (2024), Cost-optimal hybrid VM–Serverless deployment for web applications, in ‘HotInfra 2024’.
- He, Z. (2020), Novel container cloud elastic scaling strategy based on kubernetes, in ‘2020 IEEE 5th Information Technology and Mechatronics Engineering Conference (ITOEC)’, pp. 1400–1404.
- Kavis, M. (2014), ‘Cloud services and division of responsibilities’.
- Król, M. et al. (2023), HyPA: Hybrid predictive autoscaling with change-point detection, in ‘IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)’.

- Li, X. et al. (2026), ‘BACC: Budget-aware calibration and control for autoscaling’, ArXiv abs/2606.20575.
URL: <https://arxiv.org/abs/2606.20575>
- Liu, S. et al. (2024), POBO: Safe constrained Bayesian optimization for resource management, in ‘ACM SIGMETRICS’.
- Malathi, M. (2011), Cloud computing concepts, in ‘2011 3rd International Conference on Electronics Computer Technology’, pp. 236–239.
- Mampage, A., Karunasekera, S. and Buyya, R. (2022), ‘A holistic view on resource management in serverless computing environments: Taxonomy and future directions’, ACM Computing Surveys 54(11s).
- Mell, P. M. and Grance, T. (2011), The nist definition of cloud computing, Technical report, National Institute of Standards and Technology.
- Mondal, S. K., Wu, X., Kabir, H. D., Dai, H.-N., Ni, K., Yuan, H. and Wang, T. (2023), ‘Toward optimal load prediction and customizable autoscaling scheme for kubernetes’, Mathematics 11(12).
- Pahl, C., Brogi, A., Soldani, J. and Jamshidi, P. (2019), ‘Cloud container technologies: A state-of-the-art review’, IEEE Transactions on Cloud Computing 7(3), 677–692.
- RLScale-Bench: Benchmarking Deep Reinforcement Learning for Kubernetes Autoscaling (2026), ArXiv abs/2605.26418.
URL: <https://arxiv.org/abs/2605.26418>
- Sadaqat, M., Colomo-Palacios, R. and Knudsen, L. E. (2018), ‘Serverless computing: a multivocal literature review’.
URL: <https://api.semanticscholar.org/CorpusID:86762715>
- Savage, N. (2018), ‘Going serverless’, Communications of the ACM 61, 15–16.
- Senjab, K., Abbas, S., Ahmed, N. and Khan, A. u. R. (2023), ‘A survey of kubernetes scheduling algorithms’, Journal of Cloud Computing 12(1).

- Sewak, M. and Singh, S. (2018), Winning in the era of serverless computing and function as a service, in ‘2018 3rd International Conference for Convergence in Technology (I2CT)’, pp. 1–5.
- Smit, M. (2013), Supporting software evolution to the multi-cloud with a cross-cloud platform, in ‘2013 IEEE 7th International Symposium on the Maintenance and Evolution of Service-Oriented and Cloud-Based Systems’, pp. 85–85.
- Wagner, P. F. et al. (2020), ‘SeBS: Serverless benchmark suite for FaaS’, ArXiv abs/2012.14132.
URL: <https://arxiv.org/abs/2012.14132>
- Wu, Y. et al. (2025), ‘PulseNet: A serverless control plane for dual-track traffic routing’, ArXiv abs/2505.24551.
URL: <https://arxiv.org/abs/2505.24551>
- Wurster, M., Breitenbücher, U., Kepes, K., Leymann, F. and Yussupov, V. (2018), Modeling and automated deployment of serverless applications using toasca, in ‘2018 IEEE 11th Conference on Service-Oriented Computing and Applications (SOCA)’, pp. 73–80.
- Yadav, A. K., Garg, M. L. and Ritika (2019), Docker containers versus virtual machine-based virtualization, in ‘Advances in Intelligent Systems and Computing’, Springer Singapore, pp. 141–150.
- Yang, Y., Zhao, L., Li, Z., Nie, L., Chen, P. and Li, K. (2019), ElaX: Provisioning resource elastically for containerized online cloud services, in ‘Proceedings - 21st IEEE International Conference on High Performance Computing and Communications, 17th IEEE International Conference on Smart City and 5th IEEE International Conference on Data Science and Systems, HPCC/SmartCity/DSS 2019’, pp. 1987–1994.
- Yu, T., Liu, Q., Du, D., Xia, Y., Zang, B., Lu, Z., Yang, P., Qin, C. and Chen, H. (2020), Characterizing serverless platforms with ServerlessBench,

in ‘SoCC 2020 - Proceedings of the 2020 ACM Symposium on Cloud Computing’, pp. 30–44.

Yuan, H. and Liao, S. (2024), ‘A time series-based approach to elastic kubernetes scaling’, *Electronics* 13(2), 285.

Zhang, X., Li, L., Wang, Y., Chen, E. and Shou, L. (2021), ‘Zeus: Improving resource efficiency via workload colocation for massive Kubernetes clusters’, *IEEE Access* 9, 105192–105204.

LAMPIRAN

LAMPIRAN A: KONFIGURASI EKSPERIMEN

Parameter Kalibrasi

Tabel berikut menyajikan parameter kalibrasi yang digunakan dalam eksperimen, bersumber dari `CalibrationConfig (libs/shared/shared/models/calibration.py)`.

Parameter	Nilai
fib_n	33
io_wait_ms	50,0
lambda_compute_ms	24,0
r_saturation_per_replica	66,7
target_cpu_util	0,5
min_k8s_replicas	3
max_k8s_replicas	10
pod_cpu_millicores	300
node_cpu_limit	1,0
proactive_trend_threshold	3,0
proactive_approach_ratio	0,6
proactive_lookahead_steps	5

Alokasi Sumber Daya Kluster

Komponen	CPU (Docker)	Pod/Replica
K8s server node	3,0	—
K8s agent node	3,0	—
Serverless agent node	3,0	—
Dynamic workload nodes	1,0 each	—
K8s baseline pods	—	3 × 300m
Knative pods (KPA)	—	3 (min-scale)

Parameter Trace ClarkNet

Parameter	Nilai
Jumlah tahap	40
Durasi per tahap	30 detik
Total durasi	1.200 detik (20 menit)
RPS minimum	22
RPS maksimum	164
RPS rata-rata	73
Jumlah request per run	87.840

LAMPIRAN B: HASIL EKSPERIMEN MENTAH

Hasil Per-Run (n=5)

Tabel berikut menyajikan hasil individu untuk setiap run pada eksperimen `fib33_proactive` (S3, S4) dan `fib33_n5` (S1, S2).

Skenario	Run	p99 (ms)	p95 (ms)	SLO	Biaya (\$)
S1	1	112	77	43	0,091
S1	2	116	77	69	0,092
S1	3	112	77	58	0,091
S1	4	99	75	30	0,091
S1	5	110	78	6	0,091
S2	1	77	75	8	0,191
S2	2	79	75	4	0,189
S2	3	78	75	9	0,191
S2	4	78	76	8	0,192
S2	5	77	75	6	0,189
S3	1	447	233	3.533	0,069
S3	2	143	86	191	0,069
S3	3	389	167	3.818	0,070
S3	4	178	108	522	0,069
S3	5	390	200	5.963	0,069
S4	1	184	102	605	0,072
S4	2	189	94	701	0,072
S4	3	199	104	857	0,072
S4	4	224	92	1.093	0,072
S4	5	143	86	253	0,073

Output Uji Statistik

Metrik	p99 (ms)	SLO
Welch t-statistic	-1,919	-1,922
Welch p-value	0,1216	0,1247
Mann-Whitney U	9,0	9,0
Mann-Whitney p	0,5476	0,5476
Cohen's d	-1,214	-1,215
95% CI batas bawah	-229,35	-3995,60
95% CI batas atas	-10,33	-239,80

LAMPIRAN C: CODE LISTINGS

Proactive Routing (V3 Controller)

Berikut adalah potongan kode mekanisme proactive routing pada `algorithm1_v3.py`:

```
1 # Track actual load for real-time trend detection
2 self._load_history.append(current_load or 0)
3 if len(self._load_history) > 5:
4     self._load_history.pop(0)
5
6 if prediction and confidence >= threshold:
7     raw_pred = prediction.get("predicted_requests", 0)
8     if len(self._load_history) >= 3:
9         n = len(self._load_history)
10        load_trend = (self._load_history[-1] - self._load_history
11                     [0]) / (n - 1)
12        approach_ratio = current_load / k8s_capacity
13
14        if load_trend > 3.0 and approach_ratio > 0.6:
15            # Extrapolate forward: current + trend * lookahead
16            amplified = current_load + load_trend * 5 # 75s ahead
17            predicted_upper = max(raw_pred, amplified)
```

Listing 1 Proactive routing berbasis tren beban aktual

Kapasitas Efektif K8s

```
1 def _compute_k8s_capacity(self, available_replicas):
2     return max(0, available_replicas) * self.
   r_effective_per_replica
```



```

3
4 # r_effective = r_saturation * target_cpu_util
5 # = 66.7 * 0.5 = 33.35 RPS/pod
6 # 3 pods * 33.35 = 100.05 RPS total capacity
7
8 def _compute_routing_split(self, total_load, k8s_capacity):
9     if total_load <= k8s_capacity:
10         return 100, 0, False # 100% K8s
11     burst_ratio = (total_load - k8s_capacity) / total_load
12     knative_pct = burst_ratio * 100 # e.g., 30% = 30 Knative
    weight

```

Listing 2 Perhitungan kapasitas K8s dan burst ratio

CalibrationConfig

```

1 class CalibrationConfig(BaseModel):
2     fib_n: int = 33
3     io_wait_ms: float = 50.0
4     r_saturation_per_replica: float = 66.7 # Measured: fib(33)
    capacity test
5     target_cpu_util: float = 0.5 # cap = 3 * 66.7 * 0.5 = 100 RPS
6     min_k8s_replicas: int = 3
7     proactive_trend_threshold: float = 3.0 # RPS/step to trigger
8     proactive_approach_ratio: float = 0.6 # 60% of cap
9     proactive_lookahead_steps: int = 5 # 75s ahead

```

Listing 3 CalibrationConfig sebagai single source of truth

BIOGRAFI PENULIS

Halaman ini sengaja dikosongkan.